Institut de Mathématiques de Toulouse, INSA Toulouse

Aggregation by boosting algorithms

Machine Learning
2024-2025

Béatrice Laurent - Philippe Besse

Outline

- Introduction
- AdaBoost algorithm
- Boosting as stagewise additive modelling
- Gradient Boosting Models (GBM)
- Extreme Gradient Boosting (XGBoost)
- Conclusion

Introduction

- Boosting was originally introduced for binary classification problems but it has been also extended to k class classification and to regression problems.
- The main idea is to combine weak classifiers (or regressors) in a way to get a powerful aggregated procedure.
- The aggregation allows to reduce the variance of the single predictors but also their bias.
- The final prediction rule is obtained as a linear combination of a recursive sequence of predictors, where each predictor is an adaptive version of the previous one, given more weight to the observations that are badly adjusted at the previous step.
- We first present the most popular boosting algorithm, introduced by Freund and Schapire (1996), called AdaBoost for binary classification.

Outline

- Introduction
- AdaBoost algorithm
- Boosting as stagewise additive modelling
- Gradient Boosting Models (GBM)
- Extreme Gradient Boosting (XGBoost)
- Conclusion

AdaBoost algorithm

- Adaboost (or adaptive boosting) is a boosting method introduced by Freund and Schapire (1996) to combine several classifiers $f_1, \ldots, f_p$.
- It is devoted to binary classification where the output $Y \in \{-1, 1\}$.
- The principle of the algorithm is to minimize the empirical risk for the exponential loss function over the linear space $\mathcal{F}$ generated by the classifiers $f_1, \ldots, f_p$.
- The aim is to compute

$$\hat{f} = \underset{f \in \mathsf{span}(f_1, \ldots, f_p)}{\operatorname{argmin}} \left\{ \frac{1}{n} \sum_{i=1}^{n} \exp(-Y_i f(X_i)) \right\}. \tag{1}$$

AdaBoost algorithm

- Let us recall that it is convenient to consider a convex loss function ℓ satisfying the condition $\ell(z) \geq \mathbb{1}_{z<0}$, which allows to give an upper bound for the misclassification probability for the prediction rule given by the sign of $f(X)$.

$$\mathbb{E}(\ell(Yf(X))) \geq \mathbb{E}(\mathbb{1}_{Yf(X)<0}) = \mathbb{P}(Y \neq sign(f(X))).$$

- Classical convex losses ℓ are
  - the hinge loss $\ell(z) = (1-z)_+$, (used for SVM's for example),
  - the exponential loss $\ell(z) = \exp(-z)$,
  - the logit loss $\ell(z) = \log_2(1+\exp(-z))$.

AdaBoost algorithm

- Since the optimization problem (1) is complex, in order to approximate the solution, Adaboost computes a recursive sequence of predictors $\hat{f}_m$ for $m = 0, \dots M$ with

$$\begin{aligned} \hat{f}_0 &= 0 \\ \hat{f}_m &= \hat{f}_{m-1} + \beta_m f_{j_m} \end{aligned}$$

where (β_m, j_m) minimizes the empirical risk associated to the exponential loss function :

$$(\beta_m, j_m) = \underset{\beta \in \mathbb{R}, j=1,\dots,p}{\operatorname{argmin}} \left\{ \frac{1}{n} \sum_{i=1}^{n} \exp(-Y_i(\hat{f}_{m-1}(X_i) + \beta f_j(X_i))) \right\}. \quad (2)$$

AdaBoost algorithm

- At each step, the algorithm looks for the best classifier f_{j_m} in the available collection $f_1, \ldots, f_p$ and the best coefficient β_m such that adding $\beta_m f_{j_m}$ to the previous predictor $\hat{f}_{m-1}$ minimizes the empirical exponential loss.
- The final classification rule is given by

$$\hat{f}(x) = \text{sign}(\hat{f}_M(x)).$$

- M is a parameter of the procedure.

The aim of the following exercise is to compute the solution of (2).

AdaBoost algorithm

Exercise : We assume that for all $j = 1, \ldots, p$, $f_j \in \{-1, 1\}$. We denote

$$w_i^{(m)} = \frac{1}{n} \exp(-Y_i \hat{f}_{m-1}(X_i))$$

and we assume that for all $j = 1, \ldots, p$,

$$err_m(f_j) = \frac{\sum_{i=1}^n w_i^{(m)} \mathbb{1}_{f_j(X_i) \neq Y_i}}{\sum_{i=1}^n w_i^{(m)}} \in]0, \frac{1}{2}[,$$

(otherwise we change f_j into $-f_j$).
Prove that

$$j_m = \underset{j \in \{1, \ldots, p\}}{\operatorname{argmin}} \left(err_m(f_j) \right),$$

and

$$\beta_m = \frac{1}{2} \log \left(\frac{1 - err_m(f_{j_m})}{err_m(f_{j_m})} \right).$$

AdaBoost algorithm

- Note that the initial weights are equal for all the observations : $w_i^{(1)} = 1/n$ for $i = 1, \ldots, n$.
- The AdaBoost algorithm then attributes more weights in the computation of the predictor at step m to the observations for which the exponential loss $\exp(-Y_i \hat{f}_{m-1}(X_i))$ of the previous estimator is high.

AdaBoost algorithm

This leads to the AdaBoost algorithm :

- Choose a parameter M.
- $w_i^{(1)} = 1/n$ for $i = 1, \ldots, n$.
- For $m = 1, \ldots, M$
  - $j_m = \underset{j=1,\ldots,p}{\operatorname{argmin}} \left(err_m(f_j) \right)$
  - $\beta_m = \frac{1}{2} \log \left(\frac{1 - err_m(f_{j_m})}{err_m(f_{j_m})} \right)$
  - $w_i^{(m+1)} = w_i^{(m)} \exp(-Y_i \beta_m f_{j_m}(X_i))$ for $i = 1, \ldots, n$.
- $\hat{f}_M(x) = \sum_{m=1}^{M} \beta_m f_{j_m}(x)$.
- $\hat{f}(x) = \text{sign}(\hat{f}_M(x))$.

AdaBoost algorithm

- The introduction of the exponential loss is motivated by computational reasons in the context of a sequentially additive modeling approach : it leads to the simple reweighting AdaBoost algorithm.
- One can wonder about the relevance of this exponential loss function.
- The aim of the following exercise is to show that minimizing the exponential loss (at the population level) leads to the Bayes classifier.
- Although this is not true for the empirical loss, this justifies the use of the exponential loss.

Exercise : Let $f^*(x) = \underset{f}{\operatorname{argmin}} \mathbb{E}\left(e^{-Yf(X)}/X = x\right)$. Prove that

$$f^*(x) = \frac{1}{2}\log\left(\frac{\mathbb{P}(Y = 1/X = x)}{\mathbb{P}(Y = -1/X = x)}\right).$$

Deduce that $sign(f^*(x))$ corresponds to the Bayes classifier.

Outline

- Introduction
- AdaBoost algorithm
- Boosting as stagewise additive modelling
- Gradient Boosting Models (GBM)
- Extreme Gradient Boosting (XGBoost)
- Conclusion

Boosting as stagewise additive modelling

The boosting relies on an additive model based on a set of elementary basis functions of the form

$$\hat{f}_M(x) = \sum_{m=1}^{M} \beta_m b(x, \gamma_m),$$

where $(\beta_m)_{1 \leq m \leq M}$ are real coefficients and $x \mapsto b(x, \gamma)$ is a function of the multivariate input variable x, depending on a set of parameters γ. This can be a weak classifier for classification problem, a quite simple tree for regression or classification purposes, a perceptron with a single hidden layer ... Given a loss function ℓ, the goal is to solve the following optimization problem :

$$\min_{\{\beta_m, \gamma_m\}_{m=1}^{M}} \sum_{i=1}^{n} \ell(Y_i, \sum_{m=1}^{M} \beta_m b(X_i, \gamma_m)). \tag{3}$$

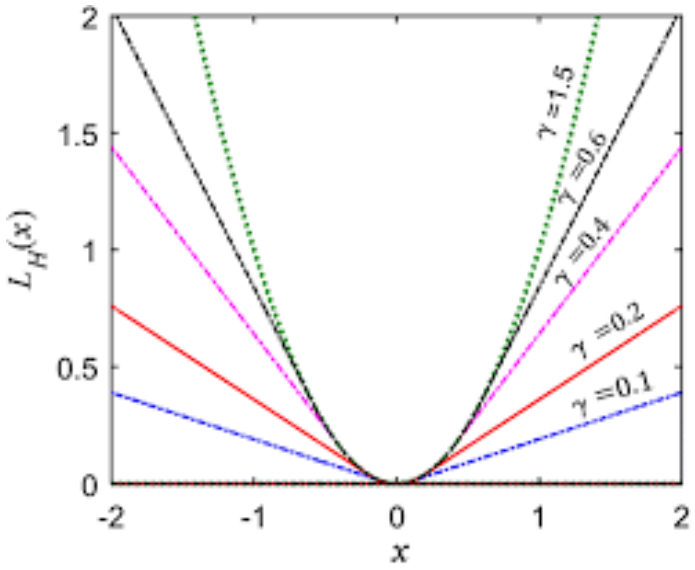
Boosting as stagewise additive modelling

- For classification purposes, the loss function ℓ can be the exponential loss, the cross-entropy (or deviance), or the logit lost.
- In the regression case, we may consider the $\mathbb{L}^2$ loss, leading to the $\mathbb{L}^2$ boosting.
- Sometimes the $\mathbb{L}^1$ loss is prefered since it is more robust (less sensitive to outliers).
- The Huber loss combines the advantages of the $\mathbb{L}^2$ loss (it is differentiable) and the $\mathbb{L}^1$ loss (robustness). It is defined by

$$\ell(y, y') = (y - y')^2 \mathbb{1}_{|y-y'| \leq \lambda} + (2\lambda|y - y'| - \lambda^2)\mathbb{1}_{|y-y'| > \lambda},$$

where λ is a positive parameter.

Huber loss



$$\psi(x) = x^2 \mathbb{1}_{|x| \leq \lambda} + (2\lambda|x| - \lambda^2)\mathbb{1}_{|x| > \lambda}$$

$$\ell(y, y') = \psi(y - y')$$

Boosting as stagewise additive modelling

- The optimization problem (3) being generally complex to solve, a step by step additive procedure allows to approximate the solution.
- This leads to the stagewise additive model, where we solve at each step

$$\begin{aligned} \hat{f}_0 &= 0 \\ \hat{f}_m &= \hat{f}_{m-1} + \beta_m b(x, \gamma_m) \end{aligned}$$

with

$$(\beta_m, \gamma_m) = \underset{(\beta,\gamma)}{\operatorname{argmin}} \left\{ \frac{1}{n} \sum_{i=1}^{n} \ell(Y_i, \hat{f}_{m-1}(X_i) + \beta b(X_i, \gamma)) \right\}. \qquad (4)$$

- The predictor $\hat{f}_m(x)$ is an improvement of the one obtained at the previous step.
- AdaBoost is a particular case of stagewise adaptive modeling with the exponential loss. Let us now consider the particular case of trees.

Boosting as stagewise additive modelling

A tree with J leaves can be expressed as

$$b(x, \gamma) = \sum_{j=1}^{J} \lambda_j \mathbb{1}_{x \in R_j},$$

with $\gamma = (\{R_j, \lambda_j\}, j = 1 \dots J)$.
A boosting tree model is a sum of such trees

$$\hat{f}_M(x) = \sum_{m=1}^{M} b(x, \gamma_m)$$

where $\hat{f}_0 = 0$, and we solve at step m :

$$\gamma_m = \underset{\gamma}{\operatorname{argmin}} \left\{ \frac{1}{n} \sum_{i=1}^{n} \ell(Y_i, \hat{f}_{m-1}(X_i) + b(X_i, \gamma)) \right\}, \tag{5}$$

$\gamma_m = (\{R_{j,m}, \lambda_{j,m}\}, j = 1 \dots J)$ are the region sets and constants corresponding to the next tree.

Boosting as stagewise additive modelling

- This problem is generally hard to solve but in some cases, it simplifies.
- In particular, if we consider a regression problem with the $\mathbb{L}^2$ loss, then one can notice that this problem is equivalent to finding a single tree to predict the residuals $Y_i - \hat{f}_{m-1}(X_i)$.

Outline

- Introduction
- AdaBoost algorithm
- Boosting as stagewise additive modelling
- Gradient Boosting Models (GBM)
- Extreme Gradient Boosting (XGBoost)
- Conclusion

Gradient Boosting Models (GBM)

- In the same spirit of stagewise additive modeling, Friedman (2002) proposed and algorithm called MART for *Multiple Additive Regression Trees*, generalized as Gradient Boosting Models.
- This algorithm is based on a differentiable loss function ℓ. The principle is still to construct a stagewise additive model.
- Each model, added to the linear combination, is a step to a better solution.
- The main innovation is that this step is done in the direction of the gradient of the loss function (assumed to be differentiable) , this gradient being itself approximated by a regression tree.
- The algorithm is described below in the regression case. It can also be adapted for classification.

Gradient Boosting Models (GBM)

Let us describe the algorithm *Gradient Tree Boosting* for regression

- Initialize $\widehat{f}_0(x) = \arg\min_\gamma \sum_{i=1}^n \ell(Y_i, \gamma)$ for all x.
- For $m = 1$ to M
  - Compute $r_{im} = -\left[\frac{\partial \ell(Y_i, f(X_i))}{\partial f(X_i)}\right]_{f=\hat{f}_{m-1}}$; $i = 1, \ldots, n$
  - Adjust a regression tree δ_m to the data $(X_i, r_{im})_{i=1,\ldots,n}$. The terminal regions are denoted $(R_{jm}, j = 1, \ldots J_m)$.
  - Compute $\gamma_{jm} = \underset{\gamma}{\operatorname{argmin}} \sum_{X_i \in R_{jm}} \ell(Y_i, \hat{f}_{m-1}(X_i) + \gamma)$.
  - Update $\widehat{f}_m(x) = \widehat{f}_{m-1}(x) + \sum_{j=1}^{J_m} \gamma_{jm} \mathbb{1}_{x \in R_{jm}}$.
- Output $\widehat{f}(x) = \widehat{f}_M(x)$.

Gradient Boosting Models (GBM)

- Note that, in the case of the $\mathbb{L}^2$ loss, setting $\ell(y, y') = \frac{1}{2}(y - y')^2$, we have
$$-r_{im} = Y_i - \hat{f}_{m-1}(X_i),$$
which means that the negative gradient is simply equal to the residuals.
- In this case, the Gradient tree boosting is equivalent to the $\mathbb{L}^2$ boosting trees presented in the previous section.
- For other losses, such that the Huber loss, this leads to new procedures.
- In a classification problem, we can consider for the loss function ℓ the deviance or the exponential loss.
- Generally, the size J_m of the tree at step m is taken to be constant for all m. This constant size J is a parameter of the procedure.

Regularization

- For boosting procedures, the number of iterations M has to be calibrated.
- Each iteration reduces the training error, hence, large values of M lead to overfitting.
- A convenient manner to calibrate this parameter is to estimate the generalization error on a validation sample, as a function of M and to choose the value of M minimizing this validation error. (This procedure is called early stopping).

Regularization

- Another way to avoid overfitting is to use a shrinkage method.
- In the case of boosting procedures, this corresponds to scaling the contribution of each tree by a factor $0 < \nu < 1$, which leads to the formula

$$\widehat{f}_m(x) = \widehat{f}_{m-1}(x) + \nu \sum_{j=1}^{J_m} \gamma_{jm} \mathbb{1}_{x \in R_{jm}}$$

  in the GBM Algorithm.
- This parameter ν corresponds to the learning rate of the gradient descent procedure.
- Reducing the value of ν increases the training error, and increases the number of iterations M.
- It seems that the best strategy is to choose a small value for ν ($\nu < 0.1$) and to calibrate M by early stopping, even if this has a computational cost.
- Generally, small trees (small value of J) with no pruning are used at each step of the boosting algorithm, which limits the computational cost.

Regularization

- It is also common to use *subsampling*, leading to a Stochastic Gradient Boosting procedure.
- At each iteration, a fraction η of the training sample is drawn without replacement, and only this subsample is used to estimate the next tree.
- This allows to reduce the computing time but also produces more accurate models, since, like for the bagging, the successive trees are less dependent, which allows to reduce the variance of the final estimator.
- Note that this procedure has also common points with batch learning for neural networks.

Outline

- Introduction
- AdaBoost algorithm
- Boosting as stagewise additive modelling
- Gradient Boosting Models (GBM)
- Extreme Gradient Boosting (XGBoost)
- Conclusion

Extreme Gradient Boosting (XGBoost)

- XGBoost algorithm was introduced by Chen and Guestrin (2016) *XGBoost : A scalable tree boosting system*
- It is based on additional penalization terms to control overfitting.
- XGBoost allows to optimize a lot of parameters, leading to a total control of the gradient Boosting algorithm.
- It also uses implementation tips for parallelization (GPU).
- The implementation is available in R (caret), Python, Julia, Spark ..

XGBoost penalization

- Chen and Guestrin (2016) "*XGBoost : A scalable tree boosting system*" propose to minimize the following objective function :

$$\sum_{i=1}^{n} \ell(f_M(X_i), Y_i) + \sum_{m=1}^{M} \Omega(f_m),$$

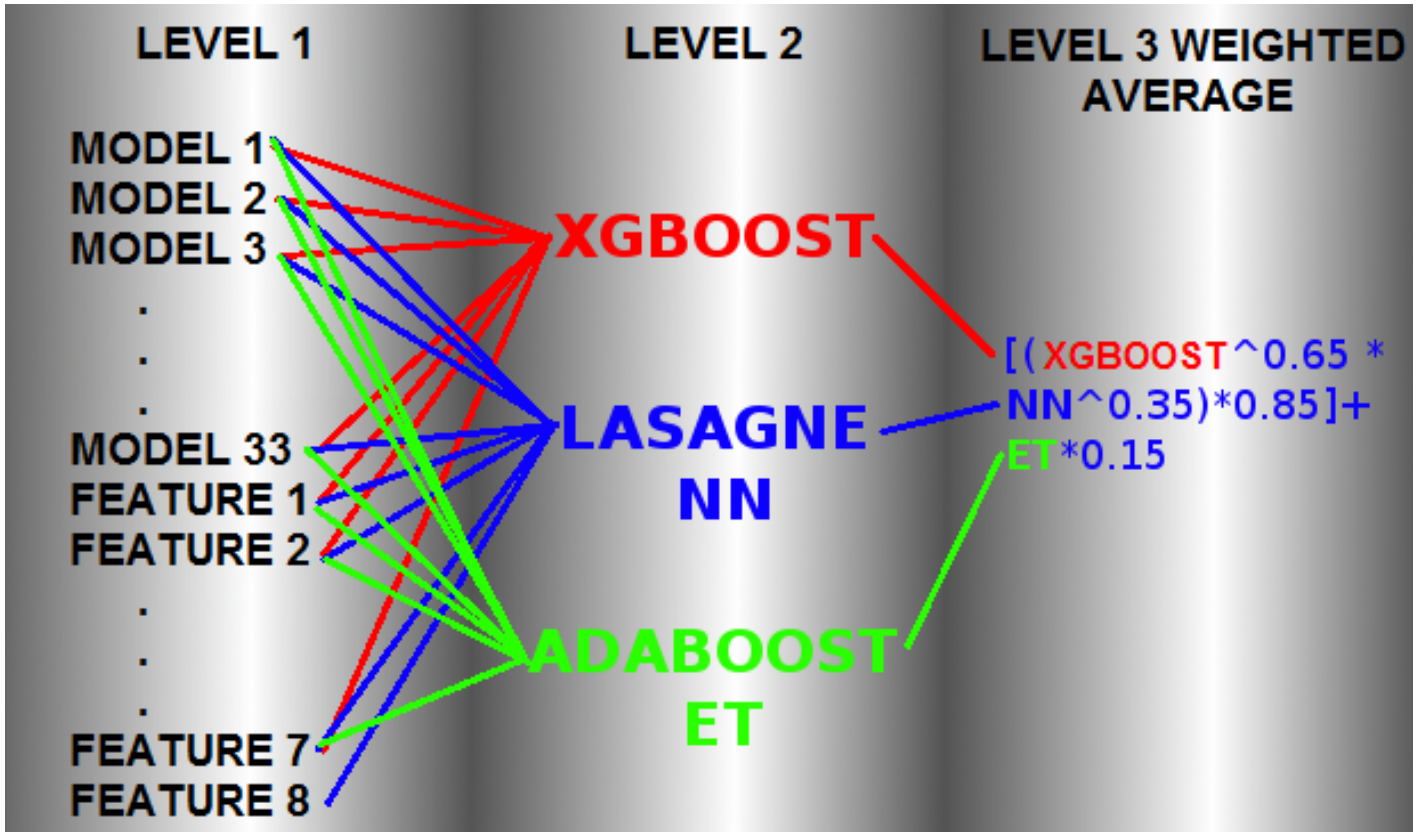
where

$$\Omega(f_m) = \gamma J_m + \lambda \|\boldsymbol{w_m}\|^2,$$

where J_m is the number of leaves of the tree f_m and $\boldsymbol{w_m}$ the vector containing the values assigned to the leaves of the tree f_m.

- The strength of XGBoost lies in the implementation tips for parallelization, making it much faster and more accurate than traditional Gradient Boosting algorithms.
- It is often the winning algorithm of Kaggle competitions.

Extreme Gradient Boosting (XGBoost)



Kaggle : Identify people who have a high degree of Psychopathy based on Twitter usage

Conclusion

- To summarize, the boosting allows to reduce the variance compared with single procedures but also the bias by aggregation, which generally leads to very performant procedures.
- Trees are easily interpretable. The interpretation is lost by aggregating. Nevertheless, like for Random Forest, indices of importance can be computed : one can average over the M trees of the boosting algorithm the indices of importance computed for each tree. This is crucial to have an idea of the relative importance of each predictor in the model.
- The Boosting is implemented in the R package `gbm` and `xgboost`. It is also implemented in the Scikit Learn library of Python (`GradientBoostingClassifier`, `GradientBoostingRegressor`, `xgboost`).

References

- Chen, T. and Guestrin, C. (2016) *XGBoost : A scalable tree boosting system*, KDD 16 : Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. https ://arxiv.org/pdf/1603.02754.pdf
- Freund, Y. and Schapire R.E. *Experiments with a new boosting algorithm*, Machine Learning : proceedings of the Thirteenth International Conference, Morgan Kaufman, 1996, San Francisco, pp. 148–156.
- Giraud C. (2015) *Introduction to High-Dimensional Statistics* Vol. 139 of Monographs on Statistics and Applied Probability. CRC Press, Boca Raton, FL.
- R.E. Shapire and Y. Freund (2012) *Boosting : Fundation and Algorithms* MIT Press.